

THE iSCALE

SQL Assignment-2

Data sets : <https://drive.google.com/drive/folders/1gHzsR-2Hi0aZrYiL1pdIGmhQMVSFbQm?usp=sharing>

Tasks:

1. Data Retrieval Techniques - GROUP BY and HAVING

Dataset: Sales

Tasks:

1. Calculate the total sales revenue for each product using GROUP BY.
 2. Filter products with revenue greater than \$1,000 using HAVING.
-

2. Dates

Dataset: Employees

Tasks:

1. Find employees who joined within the last two years.
 2. Extract the year and month from JoinDate.
-

3. SQL Joins

Dataset: Orders and Customers

Tasks:

1. Perform an INNER JOIN to get customers with orders.
 2. Use a LEFT JOIN to find customers without orders.
-

4. Join Types: Union and Self Joins

Dataset: Employees

Tasks:

1. Perform a UNION to list all employees and managers.
 2. Use a self-join to find employees and their managers.
-

5. SQL Subqueries

Dataset: Products

Tasks:

1. Find products priced above the average price.
 2. Retrieve categories with more than one product using a subquery.
-

6. Window Functions

Dataset: Sales

Tasks:

1. Calculate the running total of sales.
 2. Rank customers by their total spending.
-

7. Common Table Expressions (CTEs)

Dataset: Employees

Tasks:

1. Use a CTE to find average salary by department.
2. Use a recursive CTE to generate a sequence of numbers.

Solutions

1. Data Retrieval Techniques - GROUP BY and HAVING

Solution:

1. Total sales revenue for each product

```
SELECT Product, SUM(Quantity * Price) AS TotalRevenue
FROM Sales
GROUP BY Product;
```

2. Products with revenue greater than \$1,000

```
SELECT Product, SUM(Quantity * Price) AS TotalRevenue
FROM Sales
GROUP BY Product
HAVING TotalRevenue > 1000;
```

2. Dates

Solution:

1. Employees who joined in the last two years

```
SELECT *
FROM Employees
WHERE JoinDate >= DATEADD(YEAR, -2, GETDATE());
```

2. Extract year and month from JoinDate

```
SELECT EmployeeID, Name, YEAR(JoinDate) AS JoinYear, MONTH(JoinDate) AS
JoinMonth
FROM Employees;
```

3. SQL Joins

Solution:

1. INNER JOIN

```
SELECT o.OrderID, c.Name, o.Amount
FROM Orders o
INNER JOIN Customers c ON o.CustomerID = c.CustomerID;
```

2. LEFT JOIN

```
SELECT c.CustomerID, c.Name, o.OrderID
FROM Customers c
LEFT JOIN Orders o ON c.CustomerID = o.CustomerID
WHERE o.OrderID IS NULL;
```

4. Join Types: Union and Self Joins

Solution:

1. UNION to list employees and managers

```
SELECT EmployeeID AS PersonID, Name FROM Employees
UNION
SELECT ManagerID AS PersonID, NULL AS Name FROM Employees WHERE ManagerID
IS NOT NULL;
```

2. Self-Join to find employees and their managers

```
SELECT e.Name AS Employee, m.Name AS Manager
FROM Employees e
LEFT JOIN Employees m ON e.ManagerID = m.EmployeeID;
```

5. SQL Subqueries

Solution:

1. Products priced above the average price

```
SELECT *
FROM Products
WHERE Price > (SELECT AVG(Price) FROM Products);
```

2. Categories with more than one product

```
SELECT Category
FROM Products
GROUP BY Category HAVING COUNT(*) > 1;
```

6. Window Functions

Solution:

1. Running total of sales

```
SELECT SaleID, CustomerID, Amount,
SUM(Amount) OVER (ORDER BY SaleDate) AS RunningTotal
FROM Sales;
```

2. Rank customers by total spending

```
SELECT CustomerID, SUM(Amount) AS TotalSpent,
RANK() OVER (ORDER BY SUM(Amount) DESC) AS Rank
FROM Sales
GROUP BY CustomerID;
```

7. Common Table Expressions (CTEs)

Dataset: Employees

Tasks:

3. Use a CTE to find average salary by department.
4. Use a recursive CTE to generate a sequence of numbers.

Solution:

1. Average salary by department using CTE

```
WITH AvgSalary AS (
SELECT Department, AVG(Salary) AS AvgSalary
FROM Employees
GROUP BY Department )
SELECT * FROM AvgSalary;
```

2. Recursive CTE to generate a sequence

```
WITH Numbers AS (  
    SELECT 1 AS Num  
    UNION ALL  
    SELECT Num + 1  
    FROM Numbers  
    WHERE Num < 10 )  
SELECT * FROM Numbers;
```
